MODUL PRAKTIKUM

PRAKTIKUM DASAR PEMROGRAMAN

Modul 4

if, if-else, else-if, Nested if, dan switch

Disusun oleh:

Dr. Desta Sandya Prasvita, S.Komp., M.Kom.

PROGRAM STUDI S1 INFORMATIKA

UPN VETERAN JAKARTA

2026

Modul ini melanjutkan pemahaman mahasiswa mengenai operator relasional dan operator logika pada Modul 3. Fokus praktikum adalah membuat program yang mampu mengambil keputusan berdasarkan kondisi tertentu.

Materi ini mendukung bahan kajian logika seleksi, kondisi if, if-else, switch, dan seleksi bersarang pada RPS Algoritma dan Pemrograman.

I. Pendahuluan Modul

1. Identitas Modul

Komponen	Keterangan
Mata Kuliah	Praktikum Dasar Pemrograman
Program Studi	S1 Informatika
Semester	1
Modul	4
Judul	Pernyataan Kondisional: if, if-else, else-if, Nested if, dan switch
Bahasa Pemrograman	C
Alokasi Waktu	1 × 170 menit
Metode	Demonstrasi, latihan terbimbing, tracing, eksperimen, debugging, dan latihan mandiri
Prasyarat	Modul 1, Modul 2, dan Modul 3

2. Indikator Keberhasilan

Mahasiswa dinyatakan berhasil apabila mampu:

- menulis kondisi yang menghasilkan nilai benar atau salah;
- membedakan operator = dengan ==;
- menggunakan if, if-else, dan else-if sesuai kebutuhan;
- menentukan kondisi batas secara tepat;
- menggabungkan beberapa kondisi menggunakan &&, ||, dan !;
- menggunakan nested if tanpa membuat alur program sulit dipahami;
- membuat menu pilihan menggunakan switch;
- menggunakan break untuk mencegah eksekusi case berikutnya;
- menggunakan default untuk menangani pilihan yang tidak tersedia;
- melakukan validasi terhadap data masukan;
- memprediksi keluaran melalui trace table;
- menguji setiap kemungkinan jalur keputusan.

II. Persiapan Praktikum

1. Persiapan Folder Praktikum

Buat struktur folder berikut:

```
Praktikum_Dasar_Pemrograman/
└── Modul_04/
    ├── latihan_01.c
    ├── latihan_02.c
    ├── latihan_03.c
```

```
    ├── tracing.c
    ├── eksperimen.c
    ├── debugging.c
    └── tugas_modul_04.c
```

Ketentuan nama berkas:

- menggunakan huruf kecil;
- tidak mengandung spasi;
- menggunakan garis bawah _;
- menggunakan ekstensi .c.

III. Konsep Dasar

1. Konsep Seleksi

Program yang telah dibuat pada modul sebelumnya selalu menjalankan seluruh pernyataan secara berurutan.

Contoh:

```
printf("Masukkan nilai: ");
scanf("%d", &nilai);

printf("Nilai Anda: %d\n", nilai);
```

Pada program tersebut, seluruh instruksi dijalankan tanpa mempertimbangkan nilai yang dimasukkan.

Dalam banyak permasalahan, program perlu mengambil keputusan.

Contoh:

- apabila nilai minimal 60, mahasiswa dinyatakan lulus;
- apabila usia kurang dari 17 tahun, pengguna belum memenuhi syarat;
- apabila jumlah pembelian mencapai batas tertentu, pelanggan memperoleh diskon;
- apabila pilihan menu adalah 1, program melakukan penambahan;
- apabila input tidak valid, program menampilkan pesan kesalahan.

Struktur yang digunakan untuk mengambil keputusan disebut struktur seleksi atau percabangan.

2. Nilai Benar dan Salah dalam Bahasa C

Hasil kondisi dalam bahasa C direpresentasikan sebagai nilai numerik.

- nilai 0 dianggap salah;
- nilai selain 0 dianggap benar.

Contoh:

```
printf("%d\n", 10 > 5);
printf("%d\n", 10 < 5);
```

Keluaran:

```
1
0
```

Ekspresi pertama benar, sedangkan ekspresi kedua salah.

Contoh:

```
int nilai = 75;

printf("%d\n", nilai >= 60);
```

Keluaran:

```
1
```

3. Operator Relasional

Operator relasional digunakan untuk membandingkan dua nilai.

Operator	Arti	Contoh
==	sama dengan	nilai == 100
!=	tidak sama dengan	pilihan != 0
>	lebih besar	umur > 17
<	lebih kecil	nilai < 60
>=	lebih besar atau sama dengan	nilai >= 60
<=	lebih kecil atau sama dengan	nilai <= 100

Perbedaan = dan ==

Operator penugasan:

```
nilai = 80;
```

Artinya nilai 80 disimpan ke variabel nilai.

Operator perbandingan:

```
nilai == 80
```

Artinya program memeriksa apakah nilai sama dengan 80.

Kesalahan berikut sering terjadi:

```
if (nilai = 80)
{
    printf("Nilai adalah 80.\n");
}
```

Pernyataan tersebut bukan membandingkan nilai. Program justru memberikan nilai 80 kepada variabel nilai. Karena 80 dianggap benar, blok if akan dijalankan.

Bentuk yang benar:

```
if (nilai == 80)
{
    printf("Nilai adalah 80.\n");
}
```

Compiler dapat membantu mendeteksi kesalahan semacam ini apabila warning diaktifkan.

4. Operator Logika

Operator logika digunakan untuk menggabungkan atau membalik suatu kondisi.

Operator	Nama	Arti
&&	AND	benar apabila semua kondisi benar
||	OR	benar apabila salah satu kondisi benar
!	NOT	membalik nilai kondisi

a. Operator AND

Contoh:

```
nilai >= 0 && nilai <= 100
```

Kondisi tersebut benar apabila:

- nilai lebih besar atau sama dengan 0; dan
- nilai lebih kecil atau sama dengan 100.

Tabel sederhana:

Kondisi A	Kondisi B	A && B
Salah	Salah	Salah
Salah	Benar	Salah
Benar	Salah	Salah
Benar	Benar	Benar

b. Operator OR

Contoh:

```
pilihan == 'Y' || pilihan == 'y'
```

Kondisi benar apabila pengguna memasukkan huruf Y atau y.

Kondisi A	Kondisi B	A || B
Salah	Salah	Salah
Salah	Benar	Benar
Benar	Salah	Benar
Benar	Benar	Benar

c. Operator NOT

Contoh:

```
!(nilai >= 60)
```

Kondisi tersebut berarti nilai tidak lebih besar atau sama dengan 60.

Namun, bentuk berikut umumnya lebih mudah dibaca:

```
nilai < 60
```

Operator ! sering digunakan untuk membalik hasil suatu kondisi.

d. Gunakan tanda kurung

Kurang jelas:

```
nilai >= 0 && nilai <= 100
```

Masih benar, tetapi bentuk berikut lebih mudah dibaca:

```
(nilai >= 0) && (nilai <= 100)
```

Untuk kondisi yang lebih kompleks, penggunaan tanda kurung sangat disarankan.

5. Short-Circuit Evaluation

Bahasa C menggunakan mekanisme short-circuit evaluation pada operator && dan ||.

Operator &&

Pada ekspresi:

```
(kondisi_1) && (kondisi_2)
```

Apabila kondisi_1 sudah salah, kondisi_2 tidak perlu dievaluasi karena hasil keseluruhan pasti salah.

Contoh:

```
(pembagi != 0) && ((angka / pembagi) > 5)
```

Apabila pembagi bernilai nol, ekspresi pembagian tidak dijalankan. Hal ini mencegah pembagian dengan nol.

Operator ||

Pada ekspresi:

```
(kondisi_1) || (kondisi_2)
```

Apabila kondisi_1 sudah benar, kondisi_2 tidak perlu dievaluasi karena hasil keseluruhan sudah benar.

Short-circuit perlu dipahami, tetapi sebaiknya tidak digunakan untuk membuat kondisi yang terlalu kompleks.

6. Pernyataan if

Struktur if digunakan ketika suatu tindakan hanya dilakukan apabila kondisi bernilai benar.

Bentuk umum

```
if (kondisi)
{
    pernyataan;
}
```

Contoh:

```
int nilai = 75;

if (nilai >= 60)
{
    printf("Mahasiswa dinyatakan lulus.\n");
}
```

Karena kondisi nilai >= 60 benar, pesan ditampilkan.

Apabila nilai adalah 50, blok di dalam if tidak dijalankan.

if tanpa kurung kurawal

Bahasa C memperbolehkan:

```
if (nilai >= 60)
    printf("Lulus.\n");
```

Namun, pada praktikum ini mahasiswa diwajibkan menggunakan kurung kurawal:

```
if (nilai >= 60)
{
    printf("Lulus.\n");
}
```

Penggunaan kurung kurawal: memperjelas batas blok, mengurangi risiko kesalahan, memudahkan penambahan instruksi, dan membuat kode lebih konsisten.

7. Pernyataan if-else

Struktur if-else digunakan ketika terdapat dua kemungkinan tindakan.

Bentuk umum

```
if (kondisi)
{
    pernyataan_apabila_benar;
}
else
{
    pernyataan_apabila_salah;
}
```

Contoh:

```
int nilai = 55;

if (nilai >= 60)
{
    printf("Lulus.\n");
}
else
{
    printf("Belum lulus.\n");
}
```

Karena kondisi salah, blok else dijalankan.

Diagram alur sederhana

```
        Kondisi
       /       \
    Benar      Salah
      |           |
  Blok if     Blok else
       \         /
        Lanjut program
```

Dalam satu struktur if-else, hanya salah satu blok yang dijalankan.

8. Rangkaian else-if

Rangkaian else-if digunakan untuk menangani lebih dari dua kategori.

Bentuk umum

```
if (kondisi_1)
{
    pernyataan_1;
}
else if (kondisi_2)
{
    pernyataan_2;
}
else if (kondisi_3)
{
    pernyataan_3;
}
else
{
    pernyataan_lain;
}
```

Program memeriksa kondisi dari atas ke bawah. Ketika menemukan kondisi pertama yang benar:

- blok tersebut dijalankan;
- kondisi berikutnya tidak diperiksa;
- program keluar dari rangkaian seleksi.

Contoh klasifikasi nilai

```
if (nilai >= 85)
{
    printf("Huruf mutu: A\n");
}
else if (nilai >= 75)
{
    printf("Huruf mutu: B\n");
}
else if (nilai >= 65)
{
    printf("Huruf mutu: C\n");
}
else if (nilai >= 50)
{
    printf("Huruf mutu: D\n");
}
else
{
```

```
    printf("Huruf mutu: E\n");
}
```

Untuk nilai 78:

- nilai >= 85 salah;
- nilai >= 75 benar;
- program menampilkan B;
- kondisi berikutnya tidak diperiksa.

Urutan kondisi sangat penting

Urutan yang salah:

```
if (nilai >= 50)
{
    printf("D\n");
}
else if (nilai >= 65)
{
    printf("C\n");
}
else if (nilai >= 75)
{
    printf("B\n");
}
else if (nilai >= 85)
{
    printf("A\n");
}
```

Untuk nilai 90, kondisi pertama sudah benar karena 90 >= 50. Program menampilkan D.

Urutan yang benar menggunakan batas tertinggi terlebih dahulu:

```
if (nilai >= 85)
{
    printf("A\n");
}
else if (nilai >= 75)
{
    printf("B\n");
}
else if (nilai >= 65)
{
    printf("C\n");
}
else if (nilai >= 50)
{
    printf("D\n");
}
else
```

```
{
    printf("E\n");
}
```

9. Menentukan Batas Kondisi

Perhatikan kategori berikut:

Rentang Nilai	Huruf Mutu
85–100	A
75–84	B
65–74	C
50–64	D
0–49	E

Kondisi dapat ditulis lengkap:

```
if ((nilai >= 85) && (nilai <= 100))
{
    printf("A\n");
}
else if ((nilai >= 75) && (nilai < 85))
{
    printf("B\n");
}
```

Namun, setelah program memeriksa nilai dari batas tertinggi ke terendah, batas atas tidak selalu perlu diulang:

```
if (nilai >= 85)
{
    printf("A\n");
}
else if (nilai >= 75)
{
    printf("B\n");
}
```

Ketika blok kedua diperiksa, program sudah mengetahui bahwa nilai kurang dari 85 karena kondisi pertama gagal.

Sebelum klasifikasi, input tetap perlu divalidasi:

```
if ((nilai < 0) || (nilai > 100))
{
    printf("Nilai tidak valid.\n");
}
else if (nilai >= 85)
{
    printf("A\n");
}
```

10. Nested if

Nested if adalah struktur if yang berada di dalam blok if atau else.

Contoh

```
if (usia >= 17)
{
    if (memiliki_identitas == 1)
    {
        printf("Persyaratan lengkap.\n");
    }
    else
    {
        printf("Identitas belum tersedia.\n");
    }
}
else
{
    printf("Usia belum memenuhi persyaratan.\n");
}
```

Alurnya:

- program memeriksa usia;
- apabila usia memenuhi, program memeriksa identitas;
- apabila usia tidak memenuhi, pemeriksaan identitas tidak dilakukan.

Nested if atau operator &&

Program berikut:

```
if (usia >= 17)
{
    if (memiliki_identitas == 1)
    {
        printf("Memenuhi persyaratan.\n");
    }
}
```

dapat ditulis lebih ringkas:

```
if ((usia >= 17) && (memiliki_identitas == 1))
{
    printf("Memenuhi persyaratan.\n");
}
```

Gunakan nested if apabila:

- pemeriksaan kedua hanya relevan setelah kondisi pertama terpenuhi;
- setiap tingkat memiliki pesan atau tindakan berbeda;
- proses keputusan bersifat bertahap.

Gunakan operator logika apabila seluruh kondisi dapat diperiksa bersama dan hasil tindakannya sama.

Hindari nested if terlalu dalam

Kode berikut sulit dibaca:

```
if (kondisi_1)
{
    if (kondisi_2)
    {
        if (kondisi_3)
        {
            if (kondisi_4)
            {
                /* ... */
            }
        }
    }
}
```

Untuk mahasiswa pemula, gunakan nested if secara terbatas dan pisahkan proses menjadi beberapa tahap apabila memungkinkan.

11. Masalah Dangling else

Perhatikan program berikut:

```
if (nilai >= 60)
    if (kehadiran >= 75)
        printf("Lulus.\n");
    else
        printf("Tidak lulus.\n");
```

Secara aturan bahasa C, else akan dipasangkan dengan if terdekat yang belum memiliki else.

Artinya, else terhubung dengan:

```
if (kehadiran >= 75)
```

bukan dengan:

```
if (nilai >= 60)
```

Gunakan kurung kurawal untuk menghindari ambiguitas:

```
if (nilai >= 60)
{
    if (kehadiran >= 75)
    {
        printf("Lulus.\n");
    }
    else
    {
        printf("Kehadiran tidak memenuhi syarat.\n");
    }
}
```

```
else
{
    printf("Nilai tidak memenuhi syarat.\n");
}
```

12. Pernyataan switch

Struktur switch digunakan untuk memilih satu tindakan berdasarkan nilai suatu ekspresi.

switch sesuai untuk pilihan berupa nilai diskret, misalnya:

- nomor menu;
- kode hari;
- kode kategori;
- pilihan operasi;
- satu karakter tertentu.

Bentuk umum

```
switch (ekspresi)
{
    case nilai_1:
        pernyataan_1;
        break;

    case nilai_2:
        pernyataan_2;
        break;

    default:
        pernyataan_default;
        break;
}
```

Contoh menu

```
int pilihan;

printf("1. Tambah data\n");
printf("2. Tampilkan data\n");
printf("3. Keluar\n");
printf("Pilihan: ");
scanf("%d", &pilihan);

switch (pilihan)
{
    case 1:
        printf("Menu tambah data dipilih.\n");
        break;
```

```
    case 2:
        printf("Menu tampil data dipilih.\n");
        break;

    case 3:
        printf("Program selesai.\n");
        break;

    default:
        printf("Pilihan tidak tersedia.\n");
        break;
}
```

13. Fungsi break

Pernyataan break digunakan untuk menghentikan eksekusi suatu case.

Tanpa break:

```
switch (pilihan)
{
    case 1:
        printf("Satu\n");

    case 2:
        printf("Dua\n");

    case 3:
        printf("Tiga\n");
}
```

Apabila pilihan bernilai 1, keluaran dapat menjadi:

```
Satu
Dua
Tiga
```

Program melanjutkan eksekusi ke case berikutnya. Perilaku ini disebut fall-through.

Dalam modul dasar, setiap case umumnya harus diakhiri break, kecuali fall-through digunakan dengan sengaja dan diberi komentar.

14. Bagian default

Bagian default dijalankan apabila tidak ada case yang sesuai.

```
default:
    printf("Pilihan tidak valid.\n");
    break;
```

Meskipun tidak selalu wajib secara sintaks, default sangat disarankan untuk menangani input yang tidak tersedia.

15. Batasan switch

Ekspresi pada switch umumnya menggunakan:

- int;
- char;
- tipe enumerasi.

Contoh yang sesuai:

```
switch (pilihan)
switch (kode_hari)
```

switch tidak digunakan secara langsung untuk:

- rentang nilai;
- perbandingan seperti nilai >= 80;
- kondisi gabungan;
- nilai float atau double.

Untuk rentang nilai, gunakan if-else-if.

Contoh yang tidak dapat langsung dibuat menggunakan case:

```
Nilai 85 sampai 100 → A
Nilai 75 sampai 84  → B
```

Gunakan:

```
if (nilai >= 85)
{
    /* ... */
}
else if (nilai >= 75)
{
    /* ... */
}
```

16. Materi Pengayaan: enum

Pada bagian sebelumnya disebutkan bahwa ekspresi switch dapat menggunakan tipe enumerasi. Bagian ini memperkenalkan secara singkat apa itu enum.

enum (enumeration) adalah tipe data yang memungkinkan pemrogram memberi nama pada sekumpulan nilai konstanta bilangan bulat, sehingga kode lebih mudah dibaca dibandingkan menggunakan angka mentah (magic number).

Sintaks Dasar

```
enum Hari
{
    SENIN,
    SELASA,
    RABU,
    KAMIS,
```

```
    JUMAT,
    SABTU,
    MINGGU
};
```

Secara default, anggota pertama bernilai 0, kemudian bertambah 1 untuk setiap anggota berikutnya.

Anggota	Nilai
SENIN	0
SELASA	1
RABU	2
KAMIS	3
JUMAT	4
SABTU	5
MINGGU	6

Deklarasi variabel bertipe enum:

```
enum Hari hari_ini;

hari_ini = RABU;
```

Karena RABU bernilai 2, pernyataan tersebut secara efektif sama dengan hari_ini = 2;, tetapi jauh lebih mudah dibaca dan dipahami maksudnya.

Nilai Awal Kustom

Nilai awal dapat ditentukan secara eksplisit:

```
enum Bulan
{
    JANUARI = 1,
    FEBRUARI,
    MARET,
    APRIL
};
```

Karena JANUARI diberi nilai 1, anggota berikutnya melanjutkan urutan: FEBRUARI = 2, MARET = 3, APRIL = 4.

Kombinasi enum dengan switch

Kombinasi enum dan switch adalah salah satu penerapan enum yang paling umum, karena setiap case dapat ditulis menggunakan nama yang jelas alih-alih angka.

```
#include <stdio.h>

enum StatusPesanan
{
    DIPROSES,
    DIKIRIM,
    SELESAI,
    DIBATALKAN
};

int main(void)
```

```
{
    enum StatusPesanan status = DIKIRIM;

    switch (status)
    {
        case DIPROSES:
            printf("Pesanan sedang diproses.\n");
            break;

        case DIKIRIM:
            printf("Pesanan sedang dikirim.\n");
            break;

        case SELESAI:
            printf("Pesanan telah selesai.\n");
            break;

        case DIBATALKAN:
            printf("Pesanan dibatalkan.\n");
            break;

        default:
            printf("Status tidak dikenali.\n");
            break;
    }

    return 0;
}
```

Dibandingkan menggunakan angka 0, 1, 2, 3 secara langsung pada setiap case, penggunaan nama seperti DIPROSES dan DIKIRIM membuat program jauh lebih mudah dipahami tanpa perlu menghafal arti setiap angka.

Sekilas: enum sebagai Anggota struct

Pada praktik pemrograman, enum sering digunakan bersama struct, misalnya sebagai salah satu anggota untuk menyatakan kategori atau status suatu data. Topik struct akan dipelajari lebih lanjut pada Modul 12, dan enum dapat digunakan kembali di sana sebagai salah satu tipe data anggota struct.

17. Materi Pengayaan: Ternary Operator

Bagian ini memperkenalkan bentuk ringkas dari if-else yang disebut ternary operator atau operator kondisional.

Perhatikan pernyataan if-else sederhana berikut:

```
int a = 10;
int b = 20;
int maksimum;

if (a > b)
{
```

```
    maksimum = a;
}
else
{
    maksimum = b;
}
```

Ternary operator memungkinkan pernyataan tersebut ditulis dalam satu baris.

Sintaks

```
kondisi ? nilai_jika_benar : nilai_jika_salah;
```

Ekspresi tersebut dievaluasi sebagai berikut: jika kondisi bernilai benar, hasilnya adalah nilai_jika_benar; jika kondisi bernilai salah, hasilnya adalah nilai_jika_salah.

Contoh program if-else di atas ditulis ulang menggunakan ternary operator:

```
int a = 10;
int b = 20;
int maksimum;

maksimum = (a > b) ? a : b;
```

Karena a > b bernilai salah (10 tidak lebih besar dari 20), maka maksimum diisi dengan nilai b, yaitu 20.

Contoh: Menentukan Status Kelulusan

```
#include <stdio.h>

int main(void)
{
    double nilai = 75.0;

    printf(
        "Status: %s\n",
        (nilai >= 60.0) ? "Lulus" : "Tidak Lulus"
    );

    return 0;
}
```

Keluaran:

```
Status: Lulus
```

Kapan Ternary Operator Sesuai Digunakan

Ternary operator sesuai digunakan untuk kondisi sederhana yang hanya memilih satu dari dua nilai, terutama saat hasilnya langsung digunakan (misalnya langsung dicetak atau langsung disimpan ke variabel).

Untuk kondisi yang melibatkan beberapa pernyataan atau logika yang lebih kompleks, if-else biasa lebih sesuai karena lebih mudah dibaca.

Kurang disarankan (ternary bersarang, sulit dibaca):

```
char huruf_mutu = (nilai >= 85) ? 'A' : (nilai >= 75) ? 'B' : (nilai >= 60) ? 'C'
: 'E';
```

Untuk kasus seperti ini, if-else if bertingkat atau switch lebih mudah dibaca dan ditelusuri.

18. Materi Pengayaan: stdbool.h dan Tipe bool

Sejauh ini, seluruh kondisi pada modul ini menghasilkan nilai 1 (benar) atau 0 (salah) menggunakan tipe int, sesuai kebiasaan asli bahasa C. Bagian ini memperkenalkan cara alternatif menuliskan nilai benar/salah menggunakan tipe bool.

Bahasa C standar (sebelum C99) tidak memiliki tipe data boolean bawaan. Sejak standar C99, tipe bool tersedia melalui pustaka:

```
#include <stdbool.h>
```

Deklarasi dan Nilai

```
bool sudah_lulus = true;
bool masih_cuti = false;
```

Nilai true setara dengan 1, dan false setara dengan 0. Ketika ditampilkan menggunakan %d, true akan tercetak sebagai 1 dan false sebagai 0.

Contoh Penggunaan

```
#include <stdio.h>
#include <stdbool.h>

int main(void)
{
    int angka = 7;
    bool apakah_genap;

    apakah_genap = (angka % 2 == 0);

    if (apakah_genap)
    {
        printf("%d adalah bilangan genap.\n", angka);
    }
    else
    {
        printf("%d adalah bilangan ganjil.\n", angka);
    }

    printf("Nilai apakah_genap: %d\n", apakah_genap);

    return 0;
}
```

Keluaran:

```
7 adalah bilangan ganjil.
Nilai apakah_genap: 0
```

Perbandingan dengan Konvensi int

Aspek	Menggunakan int (dipakai pada modul ini)	Menggunakan bool (stdbool.h)
Header tambahan	Tidak perlu	Perlu #include <stdbool.h>
Nilai benar	Bilangan bukan nol, umumnya 1	true (setara 1)
Nilai salah	0	false (setara 0)
Ketersediaan	Sejak awal bahasa C	Sejak standar C99

Materi stdbool.h dan tipe bool tidak menjadi kompetensi wajib pada Modul 4. Seluruh praktikum, latihan, dan tugas pada modul ini tetap menggunakan tipe int sebagai representasi nilai benar/salah, sesuai pola pada bagian-bagian sebelumnya.

IV. Praktikum Terbimbing

1. Praktikum Terbimbing 1 — Menentukan Status Kelulusan

Buat berkas: latihan_01.c. Tuliskan program berikut:

```
#include <stdio.h>

int main(void)
{
    double nilai;

    printf("Masukkan nilai akhir: ");
    scanf("%lf", &nilai);

    if ((nilai < 0.0) || (nilai > 100.0))
    {
        printf("Nilai tidak valid.\n");
    }
    else if (nilai >= 60.0)
    {
        printf("Status: Lulus\n");
    }
    else
    {
        printf("Status: Belum lulus\n");
    }

    return 0;
}
```

Contoh pengujian

Nilai	Keluaran
-5	Nilai tidak valid
0	Belum lulus
59.99	Belum lulus
60	Lulus
100	Lulus
105	Nilai tidak valid

Pertanyaan analisis

1. Mengapa validasi dilakukan sebelum penentuan kelulusan?
2. Mengapa digunakan operator ||?
3. Apa yang terjadi apabila kondisi validasi ditempatkan setelah kondisi kelulusan?
4. Mengapa nilai 60 dinyatakan lulus?
5. Apa perbedaan nilai > 100 dan nilai >= 100?
6. Apakah 59.99 termasuk lulus?
7. Berapa banyak kemungkinan jalur keluaran dalam program tersebut?

2. Praktikum Terbimbing 2 — Menentukan Huruf Mutu

Buat berkas: latihan_02.c. Gunakan ketentuan berikut:

Rentang Nilai	Huruf Mutu
85–100	A
75–kurang dari 85	B
65–kurang dari 75	C
50–kurang dari 65	D
0–kurang dari 50	E

Program:

```
#include <stdio.h>

int main(void)
{
    double nilai;
    char huruf_mutu;

    printf("Masukkan nilai akhir: ");
    scanf("%lf", &nilai);

    if ((nilai < 0.0) || (nilai > 100.0))
    {
        printf("Nilai tidak valid.\n");
        return 1;
    }

    if (nilai >= 85.0)
    {
        huruf_mutu = 'A';
    }
    else if (nilai >= 75.0)
    {
        huruf_mutu = 'B';
    }
    else if (nilai >= 65.0)
    {
        huruf_mutu = 'C';
```

```
    }
    else if (nilai >= 50.0)
    {
        huruf_mutu = 'D';
    }
    else
    {
        huruf_mutu = 'E';
    }

    printf("Nilai akhir : %.2f\n", nilai);
    printf("Huruf mutu  : %c\n", huruf_mutu);

    return 0;
}
```

Pertanyaan analisis

1. Mengapa kondisi dimulai dari nilai tertinggi?
2. Apa hasil untuk nilai 84.99?
3. Apa hasil untuk nilai 75?
4. Apa hasil untuk nilai 49.99?
5. Apa tujuan return 1; setelah input dinyatakan tidak valid?
6. Apakah seluruh rentang nilai 0–100 telah ditangani?
7. Apakah terdapat rentang nilai yang tumpang tindih?

3. Praktikum Terbimbing 3 — Kalkulator Sederhana dengan switch

Buat berkas: latihan_03.c. Program:

```
#include <stdio.h>

int main(void)
{
    int pilihan;
    double angka_1;
    double angka_2;
    double hasil;

    printf("KALKULATOR SEDERHANA\n");
    printf("====================\n");
    printf("1. Penjumlahan\n");
    printf("2. Pengurangan\n");
    printf("3. Perkalian\n");
    printf("4. Pembagian\n");

    printf("Masukkan pilihan: ");
    scanf("%d", &pilihan);
```

```
    printf("Masukkan angka pertama: ");
    scanf("%lf", &angka_1);

    printf("Masukkan angka kedua: ");
    scanf("%lf", &angka_2);

    switch (pilihan)
    {
        case 1:
            hasil = angka_1 + angka_2;
            printf("Hasil: %.2f\n", hasil);
            break;

        case 2:
            hasil = angka_1 - angka_2;
            printf("Hasil: %.2f\n", hasil);
            break;

        case 3:
            hasil = angka_1 * angka_2;
            printf("Hasil: %.2f\n", hasil);
            break;

        case 4:
            if (angka_2 != 0.0)
            {
                hasil = angka_1 / angka_2;
                printf("Hasil: %.2f\n", hasil);
            }
            else
            {
                printf("Pembagian dengan nol tidak diperbolehkan.\n");
            }
            break;

        default:
            printf("Pilihan tidak tersedia.\n");
            break;
    }

    return 0;
}
```

Konsep yang digunakan

Program menggunakan:

- switch untuk memilih operasi;
- if-else di dalam case 4;

- validasi pembagi nol;
- break pada setiap case;
- default untuk pilihan yang tidak tersedia.

Pertanyaan analisis

1. Mengapa switch sesuai untuk program ini?
2. Mengapa pembagian membutuhkan pemeriksaan tambahan?
3. Apa yang terjadi apabila break setelah case 1 dihapus?
4. Apa yang terjadi apabila pengguna memilih 7?
5. Apakah switch dapat langsung memeriksa kondisi angka_2 != 0?
6. Mengapa if digunakan di dalam case 4?
7. Berapa banyak jalur eksekusi yang mungkin?

V. Eksperimen dan Tracing

1. Latihan Tracing

Buat berkas: tracing.c. Jangan langsung menjalankan program. Lengkapi trace table terlebih dahulu.

```
#include <stdio.h>

int main(void)
{
    int nilai = 78;
    int kehadiran = 80;
    char status;

    if (nilai >= 75)
    {
        if (kehadiran >= 75)
        {
            status = 'L';
        }
        else
        {
            status = 'T';
        }
    }
    else
    {
        status = 'T';
    }

    printf("Status: %c\n", status);

    return 0;
}
```

Keterangan: L = lulus, T = tidak lulus.

Trace table

Langkah	Kondisi/Pernyataan	nilai	kehadiran	status	Hasil Kondisi
1	nilai = 78	78	–	–	–
2	kehadiran = 80	78	80	–	–
3	nilai >= 75	78	80	–	
4	kehadiran >= 75	78	80	–	
5	status = 'L'	78	80		–

Setelah tracing:

- tuliskan prediksi output;
- jalankan program;
- ubah kehadiran menjadi 70;
- ulangi tracing;
- ubah nilai menjadi 60;
- ulangi tracing;
- bandingkan jalur yang dijalankan.

2. Eksperimen

Buat berkas: eksperimen.c. Gunakan program dasar:

```
#include <stdio.h>

int main(void)
{
    int nilai = 75;

    if (nilai >= 60)
    {
        printf("Lulus.\n");
    }
    else
    {
        printf("Belum lulus.\n");
    }

    return 0;
}
```

Lakukan eksperimen berikut satu per satu.

No.	Perubahan	Prediksi	Hasil Aktual
1	Ubah nilai >= 60 menjadi nilai > 60		
2	Ubah nilai menjadi 60		
3	Ganti >= menjadi <=		
4	Ganti == dengan = pada kondisi perbandingan		
5	Hapus kurung kurawal blok if		
6	Tambahkan dua pernyataan tanpa kurung kurawal		
7	Susun kategori nilai dari batas terendah		
8	Hapus break pada salah satu case		
9	Hapus bagian default pada switch		
10	Gunakan kondisi (nilai >= 0) && (nilai <= 100)		

Untuk setiap eksperimen:

- tuliskan prediksi;
- kompilasi dengan warning aktif;
- catat error atau warning;
- jalankan program apabila aman;
- jelaskan hasil;
- kembalikan program ke kondisi awal.

VI. Debugging Challenge

1. Debugging Challenge

Program berikut dimaksudkan untuk menentukan kategori usia dan harga tiket. Program memiliki kesalahan sintaks dan logika.

```
#include <stdio.h>

int main(void)
{
    int usia;
    int jenis_tiket;
    double harga;

    printf("Masukkan usia: ");
    scanf("%d", usia);

    printf("Jenis tiket:\n");
    printf("1. Reguler\n");
    printf("2. Premium\n");
    printf("Pilihan: ");
    scanf("%d", &jenis_tiket);

    if (usia < 0 && usia > 120)
    {
        printf("Usia tidak valid.\n");
    }
    else if (usia >= 60)
    {
        printf("Kategori: Lansia\n");
    }
    else if (usia >= 18)
    {
        printf("Kategori: Dewasa\n");
    }
    else if (usia >= 13)
    {
        printf("Kategori: Remaja\n");
    }
```

```
    else
    {
        printf("Kategori: Anak\n");
    }

    switch (jenis_tiket)
    {
        case 1:
            harga = 50000;

        case 2:
            harga = 100000;
            break;

        default:
            printf("Jenis tiket tidak valid.\n");
    }

    printf("Harga tiket: %.2f\n", harga);

    return 0;
}
```

Tugas

1. Salin program ke debugging.c.
2. Kompilasi sebelum memperbaikinya.
3. Catat error dan warning.
4. Identifikasi kesalahan input.
5. Perbaiki kondisi validasi usia.
6. Perbaiki struktur switch.
7. Pastikan harga hanya ditampilkan apabila pilihan tiket valid.
8. Uji program menggunakan beberapa data.

Beberapa kesalahan yang perlu dicari

- simbol & pada scanf;
- operator logika pada validasi usia;
- break yang hilang;
- variabel harga yang mungkin belum diberi nilai;
- alur setelah input tidak valid;
- keluaran harga pada pilihan yang salah.

Tabel debugging

No.	Bagian yang Salah	Jenis Kesalahan	Penyebab	Perbaikan
1				
2				
3				
4				
5				
6				

VII. Latihan Mandiri

1. Level 1 — Bilangan Positif, Negatif, atau Nol

Buat program yang menerima satu bilangan bulat dan menentukan apakah bilangan tersebut: positif, negatif, atau nol.

Contoh:

```
Masukkan bilangan: -8
Kategori: Bilangan negatif
```

Gunakan rangkaian if-else-if.

2. Level 2 — Bilangan Genap atau Ganjil

Buat program yang menerima satu bilangan bulat dan menentukan apakah bilangan tersebut genap atau ganjil.

Petunjuk:

```
bilangan % 2 == 0
```

Contoh:

```
Masukkan bilangan: 17
17 merupakan bilangan ganjil.
```

3. Level 3 — Validasi Nilai dan Huruf Mutu

Buat program yang menerima nilai 0–100 dan menentukan huruf mutu:

Rentang	Huruf
85–100	A
75–84.99	B
65–74.99	C
50–64.99	D
0–49.99	E

Program wajib:

- menolak nilai di bawah 0;
- menolak nilai di atas 100;
- menguji nilai batas;
- menggunakan else-if.

4. Challenge — Tahun Kabisat

Buat program untuk menentukan apakah suatu tahun merupakan tahun kabisat.

Aturan:

- tahun yang habis dibagi 400 adalah tahun kabisat;
- tahun yang habis dibagi 100 tetapi tidak habis dibagi 400 bukan tahun kabisat;
- tahun yang habis dibagi 4 tetapi tidak habis dibagi 100 adalah tahun kabisat;
- selain itu bukan tahun kabisat.

Contoh:

```
Masukkan tahun: 2024
2024 merupakan tahun kabisat.
```

Data uji penting:

Tahun	Hasil
2024	Kabisat
2023	Bukan kabisat
2000	Kabisat
1900	Bukan kabisat

5. Kategori Kecepatan Kendaraan

Keluaran program pada soal ini akan diperiksa secara ketat: harus sama persis karakter demi karakter dengan Contoh Output.

Sebuah aplikasi pemantau lalu lintas ingin mengelompokkan kendaraan berdasarkan kecepatannya. Buat program C yang menerima kecepatan kendaraan (km/jam) dan menentukan kategorinya menggunakan if-else-if.

Format Input

Satu baris bilangan bulat: kecepatan dalam km/jam.

Batasan

- $0 \leq$ kecepatan ≤ 300.

Ketentuan Kategori

Rentang Kecepatan	Kategori
kecepatan < 40	Lambat
40 ≤ kecepatan ≤ 80	Sedang
81 ≤ kecepatan ≤ 120	Cepat
kecepatan > 120	Sangat Cepat

Contoh 1

Input:

```
30
```

Output:

```
========================================
           KATEGORI KECEPATAN
========================================
```

```
Kecepatan      : 30 km/jam
Kategori       : Lambat
=======================================
```

Contoh 2

Input:

```
80
```

Output:

```
=======================================
          KATEGORI KECEPATAN
=======================================
Kecepatan      : 80 km/jam
Kategori       : Sedang
=======================================
```

Contoh 3

Input:

```
150
```

Output:

```
=======================================
          KATEGORI KECEPATAN
=======================================
Kecepatan      : 150 km/jam
Kategori       : Sangat Cepat
=======================================
```

Petunjuk Pendekatan

- gunakan variabel bertipe int untuk kecepatan;
- gunakan rangkaian if-else-if sesuai urutan tabel kategori, mulai dari kondisi kecepatan < 40;
- perhatikan Contoh 2 (kecepatan = 80) adalah nilai batas atas kategori Sedang — pastikan kondisi menggunakan <= 80, bukan < 80;
- gunakan label dengan lebar 15 karakter sebelum tanda titik dua agar sejajar, seperti pada contoh output;
- kedua baris pembatas (garis =) harus memiliki jumlah karakter yang sama seperti pada Contoh Output.

Simpan sebagai latihan_bonus_04.c.

VIII. Pengujian Program

1. Pengujian Struktur Kondisional

Program kondisional harus diuji pada setiap jalur keputusan.

a. Test case normal

Nilai yang berada di tengah suatu kategori.

Contoh: Nilai 80 → B

b. Test case batas

Nilai yang berada tepat pada batas kategori.

```
Nilai 74.99 → C
Nilai 75.00 → B
Nilai 84.99 → B
Nilai 85.00 → A
```

c. Test case tidak valid

Nilai di luar batas yang diperbolehkan.

```
Nilai -1
Nilai 101
```

Contoh tabel pengujian huruf mutu

No.	Input	Expected Output	Actual Output	Status
1	-1	Tidak valid		
2	0	E		
3	49.99	E		
4	50	D		
5	64.99	D		
6	65	C		
7	74.99	C		
8	75	B		
9	84.99	B		
10	85	A		
11	100	A		
12	101	Tidak valid		

Mahasiswa tidak cukup hanya menguji satu nilai.

IX. Tugas Praktikum

1. Tugas Praktikum Modul 4 — Program Perhitungan Tarif Parkir

Buat program C untuk menghitung tarif parkir berdasarkan jenis kendaraan, durasi parkir, dan status keanggotaan.

1. Data Input

Program menerima: jenis kendaraan, durasi parkir dalam jam, status member.

Kode jenis kendaraan:

Kode	Jenis Kendaraan
1	Sepeda motor
2	Mobil
3	Bus

Kode status member:

Kode	Status
0	Bukan member
1	Member

2. Ketentuan Tarif

Kendaraan	Jam Pertama	Setiap Jam Berikutnya	Tarif Maksimal
Sepeda motor	Rp3.000	Rp1.500	Rp15.000
Mobil	Rp5.000	Rp3.000	Rp30.000
Bus	Rp10.000	Rp5.000	Rp60.000

Ketentuan tambahan:

- durasi minimal adalah 1 jam;
- apabila tarif melampaui tarif maksimal, gunakan tarif maksimal;
- member memperoleh diskon 10%;
- diskon hanya diberikan apabila durasi parkir minimal 2 jam;
- jenis kendaraan di luar kode 1–3 dinyatakan tidak valid;
- status member selain 0 dan 1 dinyatakan tidak valid.

3. Proses Perhitungan

Apabila durasi satu jam:

```
tarif awal = tarif jam pertama
```

Apabila durasi lebih dari satu jam:

```
tarif awal =
tarif jam pertama +
(durasi – 1) × tarif jam berikutnya
```

Apabila tarif awal lebih besar daripada tarif maksimal:

```
tarif awal = tarif maksimal
```

Apabila pelanggan merupakan member dan durasi minimal dua jam:

```
diskon = tarif awal × 10%
```

Total pembayaran:

```
total bayar = tarif awal – diskon
```

4. Contoh Interaksi

```
================================================
             PROGRAM TARIF PARKIR
================================================
Jenis kendaraan:
1. Sepeda motor
2. Mobil
3. Bus
Masukkan pilihan       : 2
Masukkan durasi parkir : 5
Status member:
0. Bukan member
1. Member
Masukkan status        : 1

RINCIAN PARKIR
------------------------------------------------
Jenis kendaraan : Mobil
Durasi          : 5 jam
Tarif awal      : Rp17000.00
Diskon member   : Rp1700.00
Total bayar     : Rp15300.00
================================================
```

Perhitungan:

```
Tarif awal =
Rp5.000 + (5 – 1) × Rp3.000
= Rp17.000
```

Diskon:

```
10% × Rp17.000 = Rp1.700
```

Total:

```
Rp17.000 – Rp1.700 = Rp15.300
```

5. Struktur Kondisional yang Harus Digunakan

Program wajib menggunakan:

- switch untuk menentukan jenis kendaraan dan tarif;
- if-else untuk menghitung tarif berdasarkan durasi;
- if untuk menerapkan tarif maksimal;
- nested if atau kondisi gabungan untuk menentukan diskon;

- validasi terhadap jenis kendaraan;
- validasi terhadap durasi;
- validasi terhadap status member.

7. Analisis IPO

Mahasiswa wajib melengkapi tabel berikut sebelum menulis program.

Komponen	**Uraian**
Input	
Process	
Output	

9. Ketentuan Source Code

Program wajib:

- menggunakan #include <stdio.h>;
- menggunakan int main(void);
- menggunakan switch;
- menggunakan if-else;
- menggunakan minimal satu nested if atau kondisi gabungan;
- menggunakan operator relasional dan logika;
- menggunakan break pada setiap case;
- menggunakan bagian default;
- memvalidasi seluruh input;
- tidak melakukan perhitungan apabila input tidak valid;
- menggunakan nama variabel yang deskriptif;
- menggunakan konstanta untuk persentase diskon;
- menampilkan nominal dengan dua angka di belakang koma;
- dikompilasi dengan warning aktif;
- tidak menghasilkan error atau warning penting;
- diuji menggunakan minimal delapan test case;
- disertai trace table untuk minimal satu data uji;
- dapat dijelaskan oleh mahasiswa.

10. Header Komentar

```
/*
Nama        :
```

```
NIM         :
Kelas       :
Modul       : 4
Deskripsi   : Program menghitung tarif parkir berdasarkan
              jenis kendaraan, durasi, dan status member.
*/
```

11. Berkas yang Dikumpulkan

- source code: tugas_modul_04_nim.c
- analisis input–process–output;
- tabel keputusan;
- flowchart atau pseudocode;
- trace table;
- tabel pengujian minimal delapan kasus;
- tangkapan layar kompilasi;
- tangkapan layar hasil eksekusi;
- refleksi;
- pernyataan penggunaan AI.

2. Contoh Test Case Tugas

No.	Kendaraan	Durasi	Member	Expected Result
1	Motor	1	Tidak	Rp3.000
2	Motor	2	Tidak	Rp4.500
3	Motor	2	Ya	Rp4.050
4	Mobil	5	Ya	Rp15.300
5	Mobil	20	Tidak	Rp30.000
6	Bus	3	Tidak	Rp20.000
7	Bus	20	Ya	Rp54.000
8	Kode kendaraan 5	3	Tidak	Input tidak valid
9	Mobil	0	Tidak	Durasi tidak valid
10	Mobil	3	Status 4	Status tidak valid

Mahasiswa tetap harus menghitung sendiri expected output untuk setiap kasus.

3. Status Diskon Member

Buat program C yang menerima kode member dan harga awal, lalu menentukan besar diskon menggunakan switch dan mencetak status diskon dengan format presisi. Keluaran program dinilai secara ketat: harus sama persis karakter demi karakter, termasuk spasi.

Format Input

Dua baris: kode member (bilangan bulat) dan harga awal (bilangan bulat, rupiah).

Batasan

- $1 \leq$ harga awal ≤ 10000000;

- kode member dapat berupa bilangan bulat berapa pun (termasuk kode yang tidak terdaftar).

Ketentuan Diskon

Kode Member	Diskon
1	5%
2	10%
3	15%
Kode lainnya (default)	0%

Contoh 1

Input:

```
1
100000
```

Output:

```
=======================================
          STATUS DISKON MEMBER
=======================================
Kode member    : 1
Harga awal     : Rp100000.00
Diskon         : 5%
Harga akhir    : Rp95000.00
=======================================
```

Contoh 2

Input:

```
3
200000
```

Output:

```
=======================================
          STATUS DISKON MEMBER
=======================================
Kode member    : 3
Harga awal     : Rp200000.00
Diskon         : 15%
Harga akhir    : Rp170000.00
=======================================
```

Contoh 3

Input:

```
9
50000
```

Output:

```
=======================================
          STATUS DISKON MEMBER
```

```
======================================
Kode member    : 9
Harga awal     : Rp50000.00
Diskon         : 0%
Harga akhir    : Rp50000.00
======================================
```

Ketentuan

- gunakan switch dengan case 1, 2, 3, dan default untuk menentukan persentase diskon;
- Contoh 3 menunjukkan kode member yang tidak terdaftar (9) sehingga harus jatuh ke case default dengan diskon 0%;
- harga akhir = harga awal − (harga awal × persentase diskon);
- gunakan %.2f untuk seluruh nilai uang agar selalu tampil dua angka di belakang koma;
- kedua baris pembatas (garis =) harus memiliki jumlah karakter yang sama seperti pada Contoh Output;
- keluaran diperiksa apa adanya (karakter demi karakter), sehingga kesalahan satu spasi tetap dianggap tidak sesuai.

X. Kesalahan Umum dan Standar Penulisan

1. Kesalahan yang Sering Terjadi

a. Menggunakan = sebagai perbandingan

Salah:

```
if (pilihan = 1)
```

Benar:

```
if (pilihan == 1)
```

b. Salah memilih operator logika

Salah:

```
if ((nilai < 0) && (nilai > 100))
```

Tidak ada satu nilai yang sekaligus kurang dari 0 dan lebih dari 100.

Benar:

```
if ((nilai < 0) || (nilai > 100))
```

c. Kondisi batas tidak lengkap

Berisiko salah:

```
if (nilai > 60)
```

Apabila nilai 60 seharusnya lulus, gunakan:

```
if (nilai >= 60)
```

d. Urutan else-if salah

Salah:

```
if (nilai >= 50)
{
    huruf = 'D';
}
else if (nilai >= 85)
{
    huruf = 'A';
}
```

Nilai 90 masuk ke kondisi pertama.

e. Menggunakan beberapa if yang seharusnya saling eksklusif

Contoh:

```
if (nilai >= 50)
{
    printf("D\n");
}

if (nilai >= 65)
{
    printf("C\n");
}

if (nilai >= 75)
{
    printf("B\n");
}
```

Nilai 80 dapat menghasilkan beberapa keluaran.

Gunakan rangkaian:

```
if (nilai >= 85)
{
    printf("A\n");
}
else if (nilai >= 75)
{
    printf("B\n");
}
```

f. Tidak menggunakan kurung kurawal

Kurang aman:

```
if (nilai >= 60)
    printf("Lulus.\n");
    printf("Selamat.\n");
```

Hanya pernyataan pertama yang berada di dalam if. Pernyataan kedua selalu dijalankan.

Benar:

```
if (nilai >= 60)
{
    printf("Lulus.\n");
    printf("Selamat.\n");
}
```

g. Lupa menggunakan break

```
case 1:
    printf("Menu 1\n");

case 2:
    printf("Menu 2\n");
```

Apabila case 1 dipilih, case 2 juga dijalankan.

h. Tidak menginisialisasi variabel sebelum ditampilkan

```
double harga;

switch (pilihan)
{
    case 1:
        harga = 50000;
        break;
}

printf("%.2f\n", harga);
```

Apabila pilihan bukan 1, nilai harga belum ditentukan.

Gunakan validasi dan jangan lanjutkan program apabila pilihan salah.

i. Menggunakan switch untuk rentang

Struktur berikut tidak valid:

```
case nilai >= 85:
```

Gunakan if-else-if untuk rentang nilai.

j. Membandingkan bilangan pecahan secara langsung

Perbandingan berikut dapat bermasalah pada perhitungan pecahan tertentu:

```
if (hasil == 0.1)
```

Representasi bilangan pecahan di komputer tidak selalu tepat. Untuk Modul 4, perbandingan langsung dengan 0.0 masih digunakan pada validasi input sederhana, tetapi mahasiswa perlu mengetahui bahwa perbandingan nilai hasil perhitungan pecahan memerlukan perhatian khusus.

2. Standar Penulisan Struktur Kondisional

Gunakan indentasi yang konsisten

```
if (nilai >= 60)
{
```

```
    printf("Lulus.\n");
}
else
{
    printf("Belum lulus.\n");
}
```

Gunakan satu kondisi utama per baris apabila kompleks

```
if ((nilai >= 0.0) &&
    (nilai <= 100.0) &&
    (kehadiran >= 75.0))
{
    printf("Memenuhi syarat.\n");
}
```

Hindari kondisi negatif yang sulit dibaca

Kurang jelas:

```
if (!(nilai < 60))
```

Lebih jelas:

```
if (nilai >= 60)
```

Gunakan komentar untuk menjelaskan aturan

```
// Diskon diberikan kepada member dengan durasi minimal dua jam.
if ((status_member == 1) && (durasi >= 2))
{
    diskon = tarif_awal * TARIF_DISKON;
}
```

Komentar tidak perlu mengulang sintaks secara harfiah.

XII. Pernyataan penggunaan AI

Tidak menggunakan AI:

```
Saya menyatakan bahwa tugas ini dikerjakan tanpa menggunakan bantuan
AI generatif.
```

Menggunakan AI:

```
Saya menggunakan [nama perangkat AI] untuk membantu menjelaskan
[kondisi, error, test case, atau bagian lain]. Hasil tersebut saya
periksa melalui tracing, kompilasi, pengujian nilai batas, dan
perbandingan dengan aturan pada soal.
```

Daftar Pustaka

Deitel, P., & Deitel, H. (2016). C: How to program (8th ed.). Pearson Education.

Free Software Foundation. (n.d.). GCC, the GNU Compiler Collection: Online documentation. GNU Project. Diakses dari https://gcc.gnu.org/onlinedocs/

International Organization for Standardization. (2018). ISO/IEC 9899:2018 — Information technology — Programming languages — C. ISO.

Kernighan, B. W., & Ritchie, D. M. (1988). The C programming language (2nd ed.). Prentice Hall.

McConnell, S. (2004). Code complete: A practical handbook of software construction (2nd ed.). Microsoft Press.

Microsoft. (n.d.). C/C++ for Visual Studio Code. Visual Studio Code Documentation. Diakses dari https://code.visualstudio.com/docs/languages/cpp